Practical Fitting Guide

SMPL 피팅 실전 가이드 & 오답노트

시행착오에서 배운 것들: 무엇이 실패했고, 왜 실패했고, 어떻게 해야 하는가

2026년 4월

실제 데이터(7대 240Hz + Vicon)에 파이프라인을 적용하면서
겪은 모든 실패와 그로부터 배운 교훈을 기록합니다.

Contents

1 피팅 전략 총론 **2**
1.1 교재에서 다룬 피팅 방법 계보 2
1.2 현재 세팅에서 최적 경로 2

2 시행착오 기록 (오답노트) **3**
2.1 카메라 보정 3
2.2 세그멘테이션 3
2.3 2D 자세 추정 4
2.4 SMPL 피팅 — 핵심 난관 4

3 올바른 피팅 전략 **6**
3.1 USB에 존재하는 SMPL 파라미터 파일 목록 6
3.2 올바른 검증 방법 6
3.3 피팅이 안 맞을 경우: 재피팅 전략 7

4 하이퍼파라미터 튜닝 가이드 **8**
4.1 SMPL 피팅 하이퍼파라미터 8
4.2 SMPL 모델 선택 8

5 방법별 비교 **10**
5.1 현재 세팅의 강점과 약점 10
5.2 다음 단계 우선순위 10

6 도구 및 코드 레퍼런스 **11**
6.1 서버 환경 11
6.2 핵심 서버 경로 11
6.3 SMPL FK 실행 코드 (서버용) 11

Chapter 1

피팅 전략 총론

1.1 교재에서 다룬 피팅 방법 계보

Table 1.1: SMPL 피팅 방법 계보와 특성

방법	핵심 아이디어	입력	출력	교재
SMPLify	2D kp → 재투영 최적화	2D	$\boldsymbol{\theta}, \boldsymbol{\beta}$	Part 2
HMR/SPIN	End-to-end 회귀	이미지	$\boldsymbol{\theta}, \boldsymbol{\beta}$	Part 2
EasyMocap	다시점 2D → SMPL-X	2D × C	$\boldsymbol{\theta}, \boldsymbol{\beta}$	Part 3
3D 직접 피팅	**3D kp → SMPL 최적화**	**3D**	$\boldsymbol{\theta}, \boldsymbol{\beta}$	**Part 4**
3DGS 자세 정제	Photometric loss로 정제	영상	$\boldsymbol{\theta}^*$	Part 5

1.2 현재 세팅에서 최적 경로

Key Idea

현재 보유한 것:
- 카메라 보정: DA3 PnP refined (검증됨)
- 세그멘테이션: SAM3 마스크 7대 × 1000프레임 (완료)
- 2D 자세: OpenPose Body25 × 7대 (완료, cam7 tracked)
- 3D 자세: DLT 삼각측량 25관절 (검증됨)
- SMPL 파라미터: USB에 여러 버전 존재 (미검증)
- **누락**: smplx 라이브러리가 로컬에 없어 FK 검증 불가

최적 경로:
1. 서버(A100)에서 smplx로 기존 파라미터의 FK를 실행하여 관절 위치 추출
2. OpenPose 3D와 비교하여 올바른 파라미터 식별
3. 필요시 서버에서 3D 직접 피팅 재실행 (smplx + PyTorch)

Chapter 2

시행착오 기록 (오답노트)

2.1 카메라 보정

실패 1: COLMAP SfM 카메라 보정

시도: COLMAP으로 7대 카메라의 intrinsic/extrinsic 추정
결과: 등록률 100% 달성했으나, GT(DA3 PnP)와 비교 시 카메라 위치 오차 50%+, 초점거리 47–62% 오차
원인:

1. 동적 장면(투수 움직임)이 SfM의 정적 장면 가정 위반
2. 7대 카메라가 동일 기종인데 COLMAP이 인식 못해 초점거리를 531–1367px로 추정 (GT: ∼790px 전체 일관)
3. Round 1 (OPENCV 8파라미터)에서 과적합 → $f_x \neq f_y$

Fix

해결: DA3 PnP refined 결과를 채택.
교훈: 동적 장면에서 COLMAP SfM은 부적합. 기존 보정 결과가 있으면 그것을 사용.
추가 작업: 280×280 → 원본 해상도 역변환 (upper_bound_resize + center_crop 추적)

2.2 세그멘테이션

실패 2: cam7 SAM3 마스크 방향 불일치

시도: 기존 SAM3 마스크를 그대로 사용
결과: cam7 마스크가 portrait(1080×1920)로 생성되어 이미지(landscape 1920×1080)와 방향 불일치
추가 문제: 투수가 아닌 다른 사람이 간헐적으로 선택됨 (cam7 시야에 3명)

Fix

해결: Tracked OpenPose(P0=투수) + SAM3를 서버에서 재실행 → 1000/1000 성공, 올바른 해상도
교훈: 다인원 장면에서 person selection이 핵심. OpenPose tracking + SAM3 text prompt "person" + 키포인트 겹침 선택.

2.3 2D 자세 추정

실패 3: ViTPose person 선택 실패

시도: ViTPose(MMPose)로 원본 이미지에서 2D 키포인트 추출
결과: cam3에서 90.9%의 프레임에서 투수가 아닌 다른 사람이 P0으로 선택됨
원인: ViTPose가 detection confidence 순으로 정렬하는데, 정적인 다른 사람의 confidence가 더 높음

Fix

시도한 해결: masked images (배경 제거)에서 ViTPose 재실행
결과: 투수만 검출되지만, OpenPose 3D 대비 jitter 2배, sudden jump 30배 → 품질 열등
교훈: 기존 OpenPose + pitcher_indices가 더 안정적. "더 좋은 모델"이 항상 더 좋은 결과를 주지 않음.
참고: MMPose 기본 모델은 RTMPose이지 ViTPose-Huge가 아니었음.

2.4 SMPL 피팅 — 핵심 난관

실패 4: EasyMocap SMPL 좌표계 불일치

데이터: CJH4/easymocap_smplx_init.npz (998프레임)
결과: OpenPose 3D MidHip $(0.12, 2.81, 1.21)$ vs SMPL Th $(0.16, 0.79, 2.71)$ → **Y축과 Z축이 뒤바뀜**
원인: EasyMocap이 2D에서 3D를 재구성하면서 다른 좌표 규약 사용. 글로벌 회전(Rh) 평균 $123°$로 좌표계 보정을 흡수.

실패 5: Manual FK로 SMPL 관절 위치 계산 실패

시도: smplx 라이브러리 없이 수동 Forward Kinematics 구현 (REST pose 뼈 길이를 임의 근사)
결과: CJH2/optimized_params, CJH3/vicon_solved, CJH4/easymocap 모두 3D 뷰어에서 비현실적 스켈레톤
원인:

1. REST pose의 뼈 길이가 **임의의 근사값**이라 실제 SMPL 토폴로지와 불일치
2. SMPL의 blend shapes $(B_S(\boldsymbol{\beta}), B_P(\boldsymbol{\theta}))$를 무시하여 vertex deformation 미반영
3. 관절 위치가 regressor $(J = \mathcal{J}\bar{T})$로 결정되는데, 단순 FK 체인으로는 정확한 위치 불가능

실패 6: 새 SMPL 피팅 (서버) — MPJPE 219.6mm

시도: OpenPose 3D → smplx + PyTorch로 직접 피팅 (4-stage coarse-to-fine)
결과: MPJPE 219.6mm (발목 391–442mm, 무릎 293–300mm)
원인 추정:

1. SMPL 모델이 SMPL_NEUTRAL.pkl (304KB)로, 전체 모델이 아닌 경량 버전일 가능성
2. $\boldsymbol{\beta}$ 최적화가 불충분 (뼈 길이가 피험자 체형과 불일치)
3. Iteration 수 부족 (1400 total)
4. 자세 사전 (E_θ)이 투구 동작에 부적합

Lesson Learned

핵심 교훈: smplx 라이브러리의 Forward Kinematics가 반드시 필요하다
SMPL 파라미터 $(\boldsymbol{\theta}, \boldsymbol{\beta}, \mathbf{R}, \mathbf{t})$에서 관절 위치를 정확히 계산하려면 **smplx 라이브러리의 forward 함수를 사용해야 한다**:

```
output = body_model(
    global_orient=global_orient,   # (N, 3)
    body_pose=body_pose,           # (N, 63) or (N, 69)
    betas=betas,                   # (1, 10)
    transl=transl                  # (N, 3)
)
joints = output.joints[:, :24, :]  # Correct 3D positions
```

수동 FK는 blend shapes, joint regressor, pose-dependent corrections를 무시하므로 **절대 정확한 결과를 줄 수 없다**.

Chapter 3

올바른 피팅 전략

3.1 USB에 존재하는 SMPL 파라미터 파일 목록

Table 3.1: T31/CJH_motioncapture에서 발견된 SMPL 파라미터

#	위치	프레임	파라미터	출처
1	CJH2/optimized_params/	125	body_pose(63), hands, jaw, expression	SMPL-X full
2	CJH2/output_gpu/params/	**998**	body_pose(63), hands, jaw, expression	**GPU 최적화**
3	CJH2/output_gpu_v3/params/	**998**	동일	GPU v3
4	CJH3/vicon_solved_v13	875	body_pose(63), betas(10)	Vicon 기반
5	CJH3/data/vicon/ (v2–v11)	875	동일	여러 버전
6	CJH4/easymocap_init	998	poses(72), Rh, Th	EasyMocap
7	4DGS/smplx_sequence	125	body_pose(63), joints(22,3)	다시점
8	**새 피팅 (서버)**	**1000**	poses(72), betas(10)	OP3D 직접

3.2 올바른 검증 방법

Key Idea

모든 파라미터 파일을 GPU 서버에서 smplx FK로 관절 위치를 추출한 뒤, OpenPose 3D와 비교하여 어느 것이 가장 정확한지 판별해야 한다.

1. 서버에 파라미터 파일 업로드
2. `smplx.create()` → `output.joints[:,:24,:]` 로 관절 위치 추출
3. OpenPose 3D와 MPJPE 계산
4. 3D 뷰어로 시각적 확인
5. 가장 낮은 MPJPE를 가진 파일 채택

Warning

로컬에서 수동 FK를 쓰지 말 것. 반드시 서버의 smplx 라이브러리를 사용할 것.
수동 FK의 REST pose 뼈 길이는 실제 SMPL 토폴로지와 다르며, blend shapes와 joint regressor를 무시하므로 관절 위치가 완전히 잘못된다.

3.3 피팅이 안 맞을 경우: 재피팅 전략

기존 파라미터가 모두 부적합할 경우, 서버에서 재피팅:

1. **SMPL-X 모델 확인**: `SMPLX_NEUTRAL.npz` 사용 (SMPL_NEUTRAL.pkl이 아닌 풀 모델)
2. **3D keypoints 직접 피팅** (Part 4 방법):

$$\min_{\boldsymbol{\theta},\boldsymbol{\beta},\mathbf{R},\mathbf{t}} \sum_{j\in\mathcal{M}} w_j \|J_j^{\mathrm{SMPL}}(\boldsymbol{\theta},\boldsymbol{\beta}) + \mathbf{t} - \mathbf{p}_j^{\mathrm{OP3D}}\|^2 + \lambda_\beta\|\boldsymbol{\beta}\|^2 + \lambda_\theta\|\boldsymbol{\theta}\|^2 \tag{3.1}$$

3. **$\boldsymbol{\beta}$ 충분히 최적화**: 뼈 길이가 피험자와 맞을 때까지 (1000+ iterations)
4. **시간 정규화 적용**: 시퀀스 전체에 가속도 정규화
5. **검증**: smplx FK로 관절 위치 추출 → OpenPose 3D와 비교 (MPJPE $<$ 50mm 목표)

Chapter 4

하이퍼파라미터 튜닝 가이드

4.1 SMPL 피팅 하이퍼파라미터

Table 4.1: SMPL 3D 피팅 핵심 하이퍼파라미터

파라미터	기본값	효과	투구 조정
λ_θ	0.001	자세 정규화 (T-pose 방향)	**0.0001** (극단적 자세 허용)
λ_β	0.01	체형 정규화 (평균 방향)	0.01 유지
λ_{smooth}	0.1	시간적 매끄러움	가속 단계: **0.01**
γ (가속도)	0.5	가속도 정규화 강도	가속 단계: **0.1**
학습률 (Stage 3)	10^{-3}	$\boldsymbol{\theta}$ 최적화 속도	10^{-3} 유지
학습률 (Stage 4)	10^{-4}	공동 정제 속도	5×10^{-5}
Iterations (total)	1400	수렴 충분성	**3000+**
배치 크기	50	GPU 메모리 사용	A100: 100 가능

Important

투구 동작의 가속 단계(30ms)에서는 λ_θ와 λ_{smooth}를 줄여야 한다. 7,000°/s의 어깨 내회전을 정규화가 억제하면 정확도가 크게 저하된다. 위상별 적응형 가중치(Part 5 제안)가 이상적이지만, 단순하게 전체 시퀀스에서 $\lambda_\theta = 0.0001$로 설정해도 효과적이다.

4.2 SMPL 모델 선택

Table 4.2: SMPL 모델 파일 비교

파일	크기	관절	비고
SMPL_NEUTRAL.pkl	304 KB	24	경량, 일부 기능 제한
SMPL_NEUTRAL_FIXED.pkl	21 MB	24	풀 모델, Chumpy 제거
SMPLX_NEUTRAL.npz	∼50 MB	127	풀 SMPL-X (손+얼굴)

Warning

서버의 `~/models/smpl/SMPL_NEUTRAL.pkl`은 **304KB 경량 모델**이다. 이것으로 피팅하면 관절 위치 정확도가 떨어질 수 있다. `EasyMocap/data/smplx/smpl/SMPL_NEUTRAL_FIXED.pkl` (21MB, 풀 모델)을 사용해야 한다.

Chapter 5

방법별 비교

5.1 현재 세팅의 강점과 약점

Table 5.1: 현재 세팅 SWOT

강점	7대 카메라 240Hz, Vicon 동시 촬영, DA3 PnP 보정 검증됨, OpenPose 3D 검증됨
약점	Vicon-RGB 시간 미동기화, 체커보드 없음, 로컬에 smplx 미설치
기회	DA3 점군 활용 가능, SAM3 마스크 완비, 서버(A100) 사용 가능
위협	SMPL FK 없이는 파라미터 검증 불가, 기존 결과의 좌표계 불일치

5.2 다음 단계 우선순위

즉시 실행할 작업 (우선순위 순)

1. **서버에서 기존 SMPL 파라미터 검증**
   CJH2/output_gpu/params (998프레임)를 서버에서 smplx FK → 관절 위치 추출 → OpenPose 3D와 MPJPE 비교. **이것이 맞으면 재피팅 불필요.**

2. **맞는 파라미터가 없으면: 서버에서 재피팅**
   SMPL_NEUTRAL_FIXED.pkl (21MB) + OpenPose 3D → 3000+ iterations.

3. **Vicon-RGB 시간 동기화**
   손목 궤적 크로스-상관으로 시간 오프셋 추정.

4. **GART 학습**
   검증된 SMPL 파라미터 + 7대 영상 → GART.

Chapter 6

도구 및 코드 레퍼런스

6.1 서버 환경

항목	값
SSH	`ssh -i <pem> elicer@central-01.tcp.tunnel.elice.io -p 25341`
GPU	NVIDIA A100 80GB
Python	3.10
PyTorch	2.5.1+cu121
smplx	설치됨 (`import smplx`)
MMPose	1.3.2 (RTMPose 기본)
SAM3	`~/sam3/`

6.2 핵심 서버 경로

```
# SMPL 모델
~/models/smpl/SMPL_NEUTRAL.pkl          # 304KB 경량()
~/EasyMocap/data/smplx/smpl/SMPL_NEUTRAL_FIXED.pkl   # 21MB 풀()
~/mocap/smplx/smplx/SMPLX_NEUTRAL.npz   # SMPL-X

# 데이터
~/results번/1/set_002/step01_frames/cam{1-7}/   # 원본프레임
~/results번/1/set_002/step02_openpose/device{1-7}/   # OpenPose 2D
~/results번/1/set_002/step02_openpose_tracked/device7/   # cam7 tracked
~/CJH6_sam3/sam3_masks/cam{1-7}/   # SAM3 마스크
~/openpose3d_data.json   # OpenPose 3D 업로드됨()
~/smpl_fitting_result/   # 새피팅결과

# 결과물
~/vitpose_masked_run/    # ViTPose masked 결과
```

6.3 SMPL FK 실행 코드 (서버용)

```python
import smplx, torch, numpy as np

# Load model (FIXED version for accuracy)
model = smplx.create(
    '~/EasyMocap/data/smplx',
    model_type='smpl',
    gender='neutral',
    batch_size=50
).cuda()

# Load parameters
d = np.load('params.npz')
go = torch.tensor(d['global_orient'], dtype=torch.float32).cuda()
bp = torch.tensor(d['body_pose'], dtype=torch.float32).cuda()
bt = torch.tensor(d['betas'], dtype=torch.float32).cuda()
tr = torch.tensor(d['transl'], dtype=torch.float32).cuda()

# Forward kinematics
with torch.no_grad():
    out = model(global_orient=go, body_pose=bp,
                betas=bt, transl=tr)
    joints = out.joints[:, :24, :].cpu().numpy()

# Compare with OpenPose 3D
# joints shape: (N, 24, 3) -- correct 3D positions
```

Listing 6.1: 서버에서 SMPL 파라미터 → 관절 위치 추출